

VolSeg Documentation; Optuna Hyperparameter Search

Volume Segmantics supports automated hyperparameter optimization using [Optuna](#), an optional add-on that searches for better training settings than manual trial-and-error typically finds. It uses Bayesian optimization via a TPE sampler, rather than searching blindly or trying every combination on a grid.

Configuration and Command Line use

After installing Optuna alongside your VolSeg installation, a new command becomes available alongside `model-train-2d` and `model-predict-2d`:

```
model-train-2d --data path_to_image_data --labels path_to_segmentation_labels. --
optuna optuna_config.yaml
```

The argument points to the file `optuna_config.yaml`: a new YAML file defining which parameters to search over and how many trials to run. When this command is run, it will search for the most ideal settings and models to use for your data including the parameters *learning rate*, *model type* and *encoder name* and other settings; these can be changes in the `search space` as per your wanted choices.

A minimal `optuna_config.yaml` looks like this:

```
study_name: "volseg_optimization" #Overall Session Name
n_trials: 30 #
seed: 42 #

search_space:
  starting_lr: #Example input for search space; Learning Rate
    type: "loguniform"
    low: 1e-5 #Starting Number
    high: 1e-3 #Ending Number

  model_type: #Example input for search space; Model Type
    type: "categorical"
    choices: ["U_Net", "U_Net_Plus_plus", "FPN"] #Choices to cycle

  encoder_name: #Example input for search space; Model Encoder Name
    type: "categorical" #
    choices: ["resnet34", "resnet50", "efficientnet-b3"] #Choices to cycle

  ...
```

Each entry in `search_space` corresponds to a parameter Optuna will vary from trial to trial. Four parameter types are supported:

Type	Use for	Example
<code>loguniform</code>	learning rates, anything spanning orders of magnitude	<code>starting_lr</code> etc.
<code>uniform</code>	weights, multipliers	<code>encoder_lr_multiplier</code> etc.
<code>int</code>	epoch counts, patience	<code>num_cyc_frozen</code> etc.
<code>categorical</code>	discrete choices	<code>model_type</code> , <code>encoder_name</code> , <code>loss_criterion</code> etc.

Once optimization finishes, the best configuration found is written to a file names `best_config_<study_name>.yaml` in the working directory. You can then use this file and its contents as a connected file for your next training run, using an adaption of the normal VolSeg commands: including the arguments `cp Optuna_Config_Settings.yaml VolSeg_Training_Settings_Path.yaml` before the typical execution parameters.

The settings of the output search will work primarily on the image and labels used in the original parameter search; it should be noted that the same image label pairs should be used in the final training command though the setting may also work on comparable image/label data.

```
cp best_config_volseg_optimization.yaml volseg-
settings/2d_model_train_settings.yaml
model-train-2d --data path_to_image_data --labels path_to_segmentation_labels.
```

Example configuration

An example search space, covering more of the available parameters, is provided in the installed `optuna_config.yaml`.

Pruning

Volume Segmantics' trainer reports its validation metric to Optuna after every epoch. Trials that are clearly underperforming compared to others at the same point are stopped early (pruned), so time is not wasted fully training configurations that were never going to be competitive. Pruning behaviour can be tuned via `n_startup_trials` and `n_warmup_steps` in the Optuna config, which control how many trials and epochs are given a guaranteed full run before pruning kicks in.

Additional Command Line options

Additional arguments can be added to the `optuna_config.yaml` when running the parameter search depending on user preference, intension or ability.

Flag	Description
<code>--n-trials N</code>	Number of trials to run, overriding <code>n_trials</code> in the Optuna config
<code>--timeout SECONDS</code>	Maximum total optimization time

Flag	Description
<code>--study-name NAME</code>	Name for the study (also controls the output filename)
<code>--storage URL</code>	Database URL (e.g. <code>sqlite:///optuna.db</code>), enabling distributed optimization across multiple processes
<code>--direction {maximize,minimize}</code>	Optimization direction (default: <code>maximize</code>)
<code>--visualize</code>	Save HTML plots of the optimization to <code>optuna_plots/</code>
<code>--verbose / -v</code>	Enable debug-level logging